

Here is a structured study plan and preliminary checklist designed to guide you step-by-step through the exercises in the [NET2300 UNIX Text Processing Lab](#).

1. Prerequisites & Concept Checklist

Before attempting the exercises, make sure you are familiar with the following commands, regular expression syntax, and UNIX concepts:

Core Regular Expression (Regex) Concepts

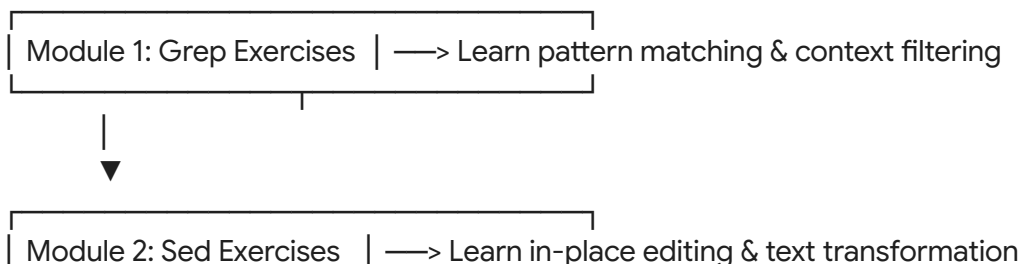
- [] **Line Anchors:** ^ (start of line) and \$ (end of line).
- [] **Character Classes:** [0-9] for digits, [^#] for non-comment characters.
- [] **Quantifiers:** ? (0 or 1 occurrence), * (0 or more), + (1 or more).
- [] **Capture Groups & Backreferences:** Using \(...\) (in basic regex) or (...) (in extended regex) to capture matched text into variables like \1, \2.

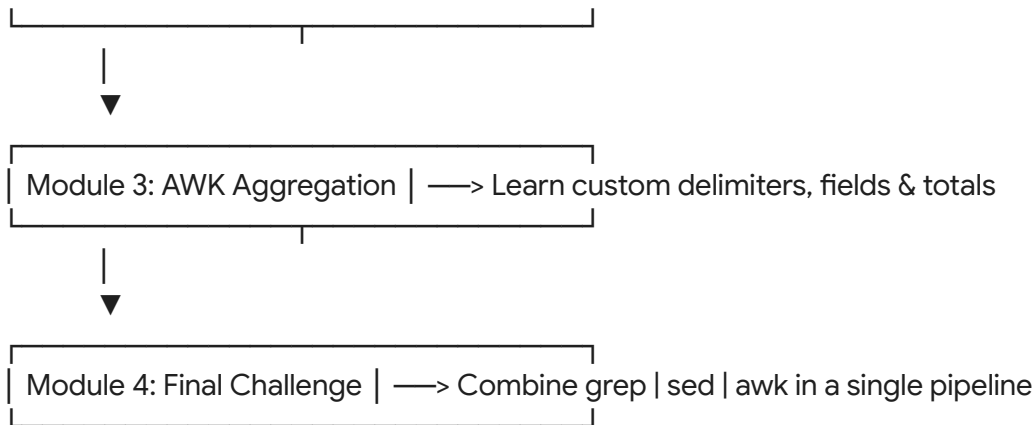
Tool-Specific Concepts

- [] **grep Basics:**
 - Context matching flags: -A (after), -B (before), -C (context).
 - Extended regex matching using -E or pcregrep / grep -P for IP matching patterns.
- [] **sed (Stream Editor) Basics:**
 - Substitutions syntax: s/pattern/replacement/g.
 - Address filtering and deletions: /pattern/d.
 - Modifying files in-place with backup creation: sed -i.bak
- [] **awk Fundamentals:**
 - Default field separators (whitespace) vs. custom delimiters (-F''').
 - Special variables: \$0 (entire record), \$1-\$N (field columns), NF (number of fields), NR (record count).
 - Control structures and special blocks: BEGIN { ... }, { body }, END { ... }.
 - Associative arrays for aggregation: count[\$1]++.
- [] **UNIX Pipelines:**
 - Piping outputs between commands using |.

2. Structured Study & Implementation Plan

Follow this sequential module-based workflow to build up your skills from pattern searching to multi-stage command pipelines.





Module 1: Pattern Search & Context (grep)

Goal: Practice isolating specific lines and extracting patterned data without modifying files.

1. Exercise 1.1 — Multi-Line Context Tracking:

- *Objective:* Find HTTP Status 401 entries in access.log.
- *Action:* Use `grep " 401 "` or `grep -C` if surrounding lines are required for context.

2. Exercise 1.2 — Strict Address Extraction:

- *Objective:* Extract only IPv4 addresses, each on a new line.
- *Action:* Combine `grep -o` (only matching parts) with an extended regular expression (`-E`) matching standard IPv4 octets (`[[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}`).

Module 2: In-Place Editing & Pattern Deletion (sed)

Goal: Master safe file modification and line-level transformation.

1. Exercise 2.1 — Safe Configuration Update with Backup:

- *Objective:* Update #Port 22 to Port 2222 in `sshd_config.mock` while creating a `.bak` file.
- *Action:* Run `sed -i.bak 's/#Port 22/Port 2222/' sshd_config.mock`.

2. Exercise 2.2 — Deterministic Policy Enforcement:

- *Objective:* Uncomment `PermitRootLogin` line and set value to `no`.
- *Action:* Match optional leading comment signs using `^#?` and replace the full line string: `sed -i 's/^#\?PermitRootLogin.*/PermitRootLogin no/' sshd_config.mock`.

3. Exercise 2.3 — Comment & Blank Line Stripping:

- *Objective:* Print active configuration lines by removing comments (`#`) and empty lines.
- *Action:* Combine delete patterns: `sed -e '/^#/d' -e '/^$/d' sshd_config.mock`.

Module 3: Aggregation & Field Parsing (awk)

Goal: Use `awk` to extract numerical fields, parse non-standard delimiters, and summarize data.

1. Exercise 3.1 — Bandwidth Accumulation:

- *Objective:* Calculate the sum of byte transfers from Field 10 in access.log.
- *Action:* Accumulate values per line and print in an `END` block: `awk '{total += $10} END {print total}' access.log`.

2. Exercise 3.2 — Structured Audit Formatting:

- *Objective:* Format uncommented configuration lines as Directive: [Name] | Value: [Val].
- *Action:* Filter out comments and empty records (`NF > 0 && !/^#/`), then output

formatted fields \$1 and \$2.

3. Exercise 3.3 — Parsing Double-Quoted Fields:

- *Objective:* Parse User-Agent strings using double quotes as separators.
- *Action:* Set field separator to double quotes (`awk -F'"'`), extract Field 6 (`$6`), and use associative arrays to tally occurrences (`count[$6]++`).

Module 4: Integrated Incident Response Pipeline Challenge

Goal: Combine tools into a single multi-stage command pipeline.

● Exercise 3.4 — Incident Response Pipeline:

- **Step 1:** Isolate requests targeting `/login.php` using `grep` or `awk`.
- **Step 2:** Extract the IP address (`$1`) and status code (`$9` or target field).
- **Step 3:** Map status code 401 to FAILED and 200 to SUCCESS using `sed` or conditional statements in `awk`.
- **Step 4:** Aggregate and output total FAILED attempts grouped per offending IP address using `awk` associative arrays.
- **Pipeline Outline:**

```
grep "/login.php" access.log | awk '{print $1, $9}' | sed 's/401/FAILED/; s/200/SUCCESS/' | awk  
'$2=="FAILED" {failed[$1]++} END {for (ip in failed) print ip, failed[ip}]'
```